

OBSERVABILIDAD EN LA TOOLCHAIN

Brian Demers habla en el episodio 675 de SE Radio de la observabilidad en la toolchain.

Demers es un reconocido desarrollador, Java Champion y miembro de Apache que contribuye a Java, Gradle y a proyectos OSS con tutoriales, blogs, documentación, ...

Antes de hablar de la observabilidad en la toolchain hay que entender que son ambas:

La toolchain es el conjunto de todas las herramientas que intervienen en el ciclo de desarrollo y entrega del software: compiladores, build tools, linters, tests, escáneres de seguridad y gestión de dependencias. No es una sola herramienta, sino todo lo que lleva el código hasta producción.

La observabilidad es la capacidad de entender qué ocurre durante el proceso de construcción en un sistema, basándose en la recolección de datos para optimización. Centrándonos en la toolchain, tiene el objetivo de acabar con la “caja negra” entre los outputs y los inputs en un proceso estándar

Generalmente la observabilidad se aplica a la producción, pero Brian propone extender esa misma mentalidad a todo el proceso de desarrollo para no solo detectar fallos, sino también explicar el por qué, dónde y la causa de estos antes de llegar a producción.

Para recolectar los datos, se usan logs (que detectan advertencias, errores y pasos concretos del pipeline), trazas (para localizar cuellos de botella dentro del build) y métricas (que estudian la evolución del sistema comparando tiempos, tasas, etc.)

» **Shift left: feedback antes y más cerca, y los problemas de no usarlo**

Shift-left significa llevar la información útil lo más cerca del desarrollador y lo antes posible, para conseguir una mayor productividad. Un ejemplo claro son los flaky tests: si el desarrollador sabe desde el inicio que un test es inestable, evita perder tiempo persiguiendo un falso problema.

Los principales problemas desde el punto de vista de observabilidad a los que se puede enfrentar un desarrollador son:

Tests flaky – pasan o fallan sin cambios de código. Generan tiempo perdido y desconfianza en el pipeline. Las métricas permiten detectar el patrón y tomar decisiones.

Ciclos de feedback lentos – builds de varios minutos dañan la fluidez, ya que cada interrupción llevará aproximadamente 20 minutos de reenfoque por el context switching (el build largo lleva al desarrollador a trabajar en otra cosa, por lo que al volver sobre lo anterior pierde el hielo). La observabilidad identifica la mala configuración que lo causa.

Problemas de dependencias –versiones duplicadas, librerías obsoletas, conflictos. Son muy frecuentes en Java y pasan fácilmente desapercibidos sin logs o alertas activas.

En el podcast también se habla de la experiencia del desarrollador, mencionando que el feedback rápido y las herramientas predecibles hacen al developer más productivo y valorado. Brian relaciona directamente la observabilidad con la developer experience: menos fricciones, más foco, más calidad

» **Por donde empezar a incorporar observabilidad en mi sistema**

Es mejor recolectar datos básicos a no tener ningún dato, por lo que Brian recalca la importancia de medir elementos básicos como tiempos o nº de warnings, además de la importancia de instrumentalizar (para comparar los datos) y crear alertas.

También defiende que esto se debe realizar de manera distinta para proyectos Greenfield (nuevos), donde importa instrumentar desde el inicio y recoger datos, aunque no sean perfectos para analizarlos y definir alertas, y los sistemas legacy (ya creados) donde lo más importante es centrarse en los datos de los problemas que ya tenemos

Una de las cosas a tener en cuenta para la observabilidad son los warnings del compilador, deprecaciones y señales similares que pasan desapercibidos porque nadie revisa miles de líneas de log tras un build exitoso. Si nadie los mira, la deuda técnica se acumula en silencio. Por eso, Brian propone convertir los warnings a errores o monitorizar su número y marcar objetivos de reducción.

» **El impacto de la IA en la observabilidad**

Como en cualquier rama de la informática, la IA tiene ventajas e inconvenientes. Con IA hay riesgos como la generación de más código inseguro y de origen desconocido, que ralentiza el build, aunque abre grandes oportunidades principalmente por su capacidad de reconocer patrones y clasificar errores, acelerando soluciones. Así, Demers plantea medir para cada caso si herramientas de IA realmente mejorarían la productividad o solo añadirían volumen

» **Incorporar la observabilidad en la toolchain al entorno empresarial**

Según Brian, las empresas deberían invertir en observabilidad por su capacidad de mejorar la toma de decisiones con el uso de los datos que se consiguen, dado que esto sería beneficioso por el ahorro de costes monetarios y de tiempo, la reducción de riesgos, la aceleración de las soluciones y la reducción del tiempo para llegar a producción relacionado con el aumento de la productividad que se conseguiría.

No obstante, resalta que el problema de la observabilidad en la actualidad también es humano, no solo técnico. Aunque se tengan muchos datos, si nadie los mira o se siente

responsable de ellos, no cambia nada, por lo que hay que fomentar la educación a los equipos y la responsabilidad compartida, adjuntando información de observabilidad.

Además, el futuro de la observabilidad también es prometedor, centrándose en la productividad y en la seguridad e incorporando mayor transparencia, automatización, políticas de confianza e incluso herramientas de IA para lograr mejores soluciones.

Como conclusión, la toolchain no debe verse como un conjunto secundario de scripts, sino como un sistema crítico que necesita observabilidad. Entender cómo compila, prueba, descarga y falla el software antes de llegar a producción mejora la calidad, la productividad, la experiencia de desarrollo y la capacidad de respuesta del equipo. Además, la facilidad de incorporar las herramientas de observabilidad es otra razón para que se convierta en una práctica extendida